

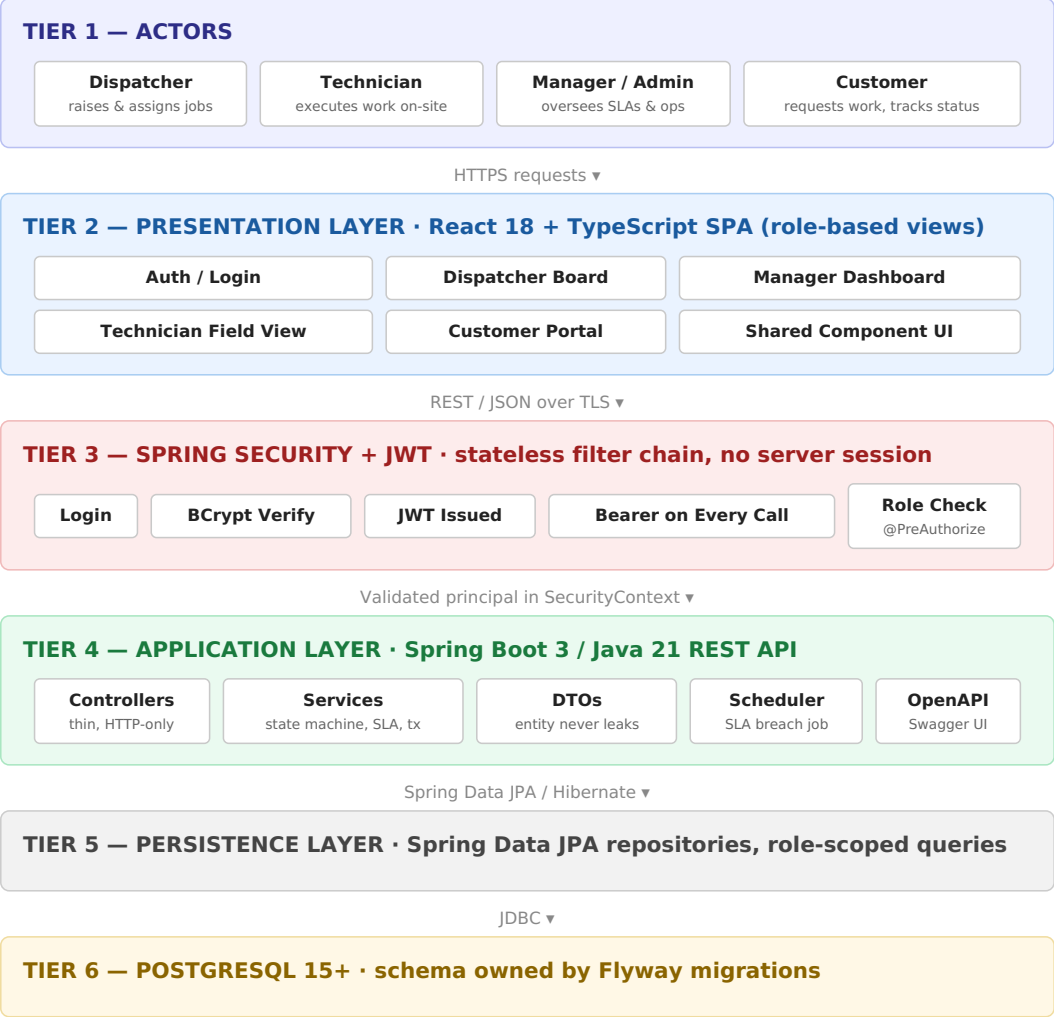
Project KEYSTONE

Schema · System Design · Architecture · Workflows · Technology Stack

Client: Meridian Facilities Management · Spring Boot 3 (Java 21) · React + TypeScript · PostgreSQL · v1.0

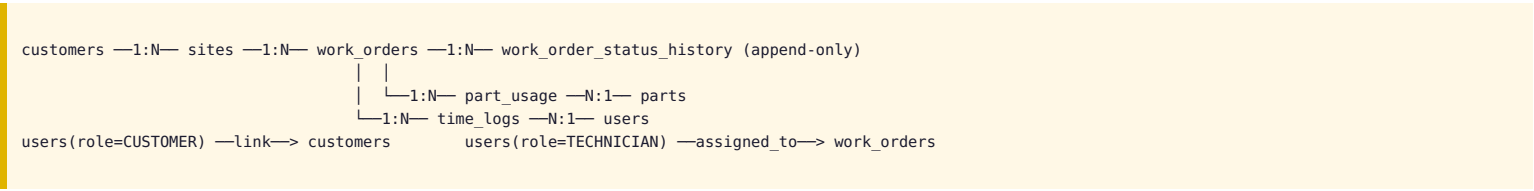
System Architecture — Layered Design

ARCH-V1.0



PostgreSQL, versioned through Flyway migrations (V1__init_schema.sql). WorkOrder is the central entity; audit and line-item tables hang off it 1:N.

Entity-relationship overview



Core tables

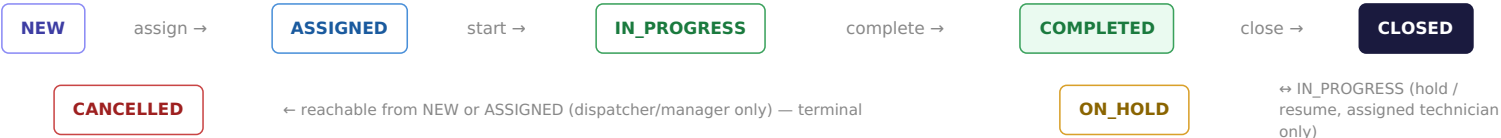
Table	Key columns	Purpose
users	id · name · email (unique) · password_hash (BCrypt) · role · customer_id (nullable) · active	Platform login for all four roles
customers	id · name · contact_email	An organisation Meridian serves
sites	id · customer_id (FK, NOT NULL) · name · address	A building where work happens
parts	id · name · sku (unique) · unit_cost · stock_qty (CHECK ≥ 0)	Inventory catalog
work_orders	id · code (unique) · title · priority · status · customer_id (FK) · site_id (FK) · assigned_to (FK, nullable) · sla_due_at · sla_breached	The unit of work — heart of the schema
work_order_status_history	id · work_order_id (FK) · from_status · to_status · changed_by (FK) · note · changed_at	Append-only audit trail; never updated/deleted
part_usage	id · work_order_id (FK) · part_id (FK) · qty_used (CHECK > 0) · unit_cost_at_use (snapshot)	Parts consumed on a job, written transactionally with stock decrement
time_logs	id · work_order_id (FK) · technician_id (FK) · minutes (CHECK > 0) · note	Labour logged against a job

Integrity rules: a work order always has customer_id + site_id (NOT NULL, FK). Status-history rows are insert-only — no service method ever updates or deletes one. Logging parts (part_usage insert + parts.stock_qty decrement) happens inside a single @Transactional method — both commit or neither does. Stock never goes negative: enforced by a DB CHECK constraint *and* a service-layer guard.

Indexes

Table	Index	Why
work_orders	status	Board/list filtering by status
work_orders	assigned_to	Technician's own queue
work_orders	customer_id, site_id	Customer-portal scoping, site drill-down
work_orders	sla_due_at (partial, open jobs only)	Fast SLA-breach scheduled scan
work_order_status_history	work_order_id	Fetch full audit trail for one job

The lifecycle is enforced centrally in WorkOrderService, never trusted from the UI. Every accepted transition writes one append-only history row in the same transaction.



Allow-list — the only transitions the service accepts

From	To	Who
NEW	ASSIGNED	Dispatcher, Manager
NEW	CANCELLED	Dispatcher, Manager
ASSIGNED	IN_PROGRESS	the assigned Technician only
ASSIGNED	CANCELLED	Dispatcher, Manager
IN_PROGRESS	ON_HOLD	the assigned Technician only
IN_PROGRESS	COMPLETED	the assigned Technician only
ON_HOLD	IN_PROGRESS	the assigned Technician only
COMPLETED	CLOSED	Manager only

Any other (from, to) pair → HTTP 409 Conflict. CLOSED and CANCELLED are terminal — no outgoing edges are permitted from either.

SLA Monitoring & Notification Flow

INPUT

Priority Set

LOW / MEDIUM / HIGH / CRITICAL
chosen at creation

CALC

SLA Due Date

created_at + offset(priority) —
4h/8h/24h/72h

SCHEDULE

Breach Scan

SlaScheduler runs every 5 min over
open work orders

EVALUATE

Flag Breach

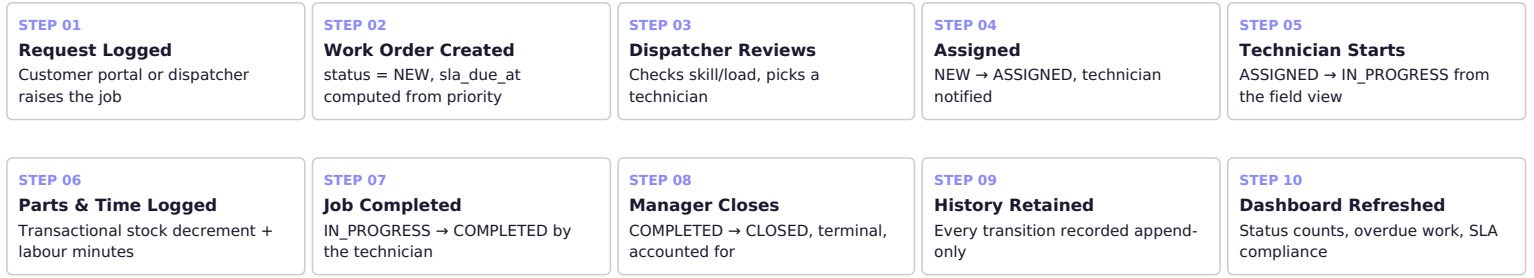
sla_due_at < now → sla_breached
= true

SURFACE

Notify & Display

manager dashboard, board badge,
notification

From an inbound request to a closed, accounted-for job.



Sample request trace — technician completes a job

```
PATCH /api/work-orders/42/status { "toStatus":"COMPLETED", "note":"Fixed compressor" }
→ JwtAuthFilter validates token → SecurityContext(userId=17, role=TECHNICIAN)
→ WorkOrderController.transitionStatus()  [@PreAuthorize technician/dispatcher/manager]
→ WorkOrderService.transition(id=42, toStatus=COMPLETED, actor=17)
  - loads WorkOrder, checks actor == assignedTo
  - checks IN_PROGRESS → COMPLETED is an allowed edge
  - saves WorkOrder + inserts WorkOrderStatusHistory  (one @Transactional method)
→ 200 OK → WorkOrderResponse DTO (id, code, status, slaDueAt, history[])
```

Role x action matrix

Action	Dispatcher	Technician	Manager	Customer
Create customer / site	✓	—	✓	—
Create work order	✓	—	✓	✓ (own site only)
Assign / reassign	✓	—	✓	—
Start / hold / resume own job	—	✓	—	—
Complete own job	—	✓	—	—
Close job	—	—	✓	—
Log parts / time on own job	—	✓	✓	—
View all work orders / board	✓	own only	✓	own only
View dashboard / reports	—	—	✓	—
Manage users / parts catalog	—	—	✓	—

Layer	Technology	Why
Language	Java 21	Modern LTS — records, pattern matching
Back-end framework	Spring Boot 3 (Web, Validation)	Industry-standard layered services
Security	Spring Security + JWT	Stateless auth, method-level role enforcement
Persistence	Spring Data JPA / Hibernate	Entity mapping and repositories
Database	PostgreSQL 15+	Reliable relational store, ACID transactions
Migrations	Flyway	Versioned, reviewable schema changes — no ddl-auto
Front end	React 18 + TypeScript (Vite)	Typed, component-based SPA
API docs	springdoc-openapi / Swagger UI	Live, browsable API reference
Build / deploy	Maven · Docker · cloud host	Reproducible builds and deployment

DEPLOYMENT

Deployment Architecture

Containerized physical topology — client access over TLS, one Docker boundary hosting three containers.



Representative API Surface

Endpoint	Method	Purpose
/api/auth/login	POST	Authenticate; returns a JWT
/api/work-orders	GET / POST	List (role-scoped, paged, filterable) or create
/api/work-orders/{id}	GET	Fetch one, with full status history
/api/work-orders/{id}/assign	POST	Assign to a technician
/api/work-orders/{id}/status	PATCH	Transition status — validated against the lifecycle, 409 on illegal jump
/api/work-orders/{id}/parts	POST	Log parts used — transactional stock decrement
/api/work-orders/{id}/time	POST	Log technician time
/api/reports/summary	GET	Dashboard metrics — status counts, overdue, SLA compliance

Status codes are conventional: 200/201 success · 400 validation · 401/403 auth · 404 missing · 409 illegal state transition. Errors always return the same JSON shape (timestamp, status, message, field errors) — never a stack trace.